AVL Tree: RL (Right-Left) Rotation

Report

1. Problem Statement

Implement an AVL tree and demonstrate how an **RL (Right-Left) imbalance** is corrected using a **double rotation**: a right rotation on the right child, followed by a left rotation on the unbalanced node. Display each step of the double rotation, print the balance factors before and after rebalancing, and test a second insertion sequence that also produces an RL imbalance.

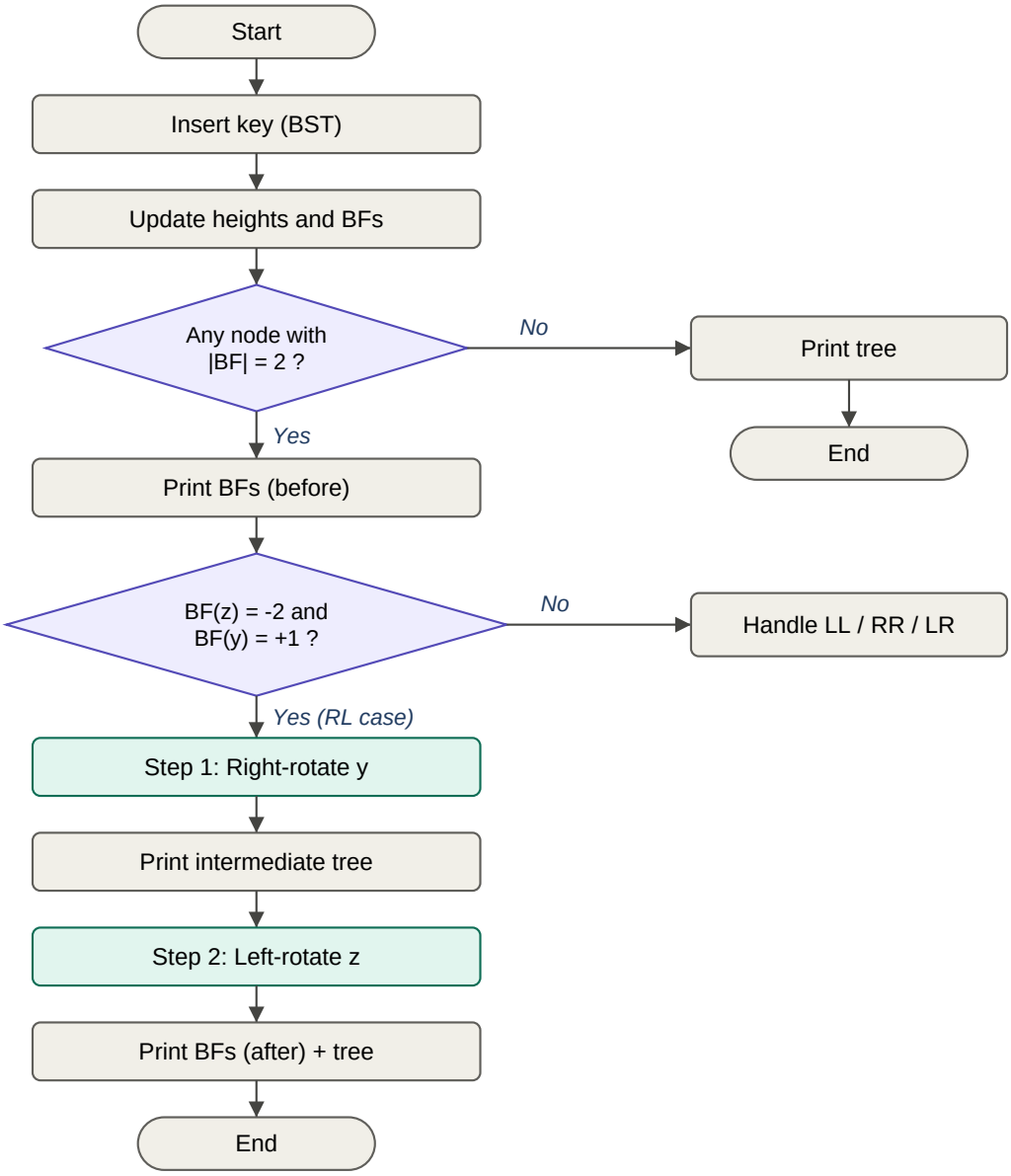
2. Objectives

- Insert keys into an AVL tree while tracking node heights.
- Detect the RL case: BF(z) = -2 and BF(y) = +1, where y = z.right.
- Perform and display each step: right-rotate y, then left-rotate z.
- Print balance factors (BF = height(left) - height(right)) before and after rebalancing.
- Verify the result with a second insertion sequence.

3. Algorithm

1. Insert the key as in a normal BST.
2. Walk back up to the root, updating the height and BF of each ancestor.
3. At the first node z with |BF| = 2, check: if BF(z) = -2 and BF(z.right) = +1, it is an RL case.
4. Print BFs of z, y = z.right and x = y.left (before).
5. **Step 1:** Right-rotate y. x becomes z's right child. Print the intermediate tree.
6. **Step 2:** Left-rotate z. x becomes the root of the subtree.
7. Print the final tree and the BFs of x, z and y (after).
8. Continue upward until the root is reached.

4. Flowchart


Start
Insert key (BST)
Update heights and BFs
Any node with
|BF| = 2 ?
No
Print tree
End
Yes
Print BFs (before)
BF(z) = -2 and
BF(y) = +1 ?
No
Handle LL / RR / LR
Yes (RL case)
Step 1: Right-rotate y
Print intermediate tree
Step 2: Left-rotate z
Print BFs (after) + tree
End

5. C++ Program

```
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;

struct Node {
    int key, height;
    Node *left, *right;
    Node(int k) : key(k), height(1), left(nullptr), right(nullptr) {}
};

int height(Node* n) { return n ? n->height : 0; }
int bf(Node* n) { return n ? height(n->left) - height(n->right) : 0; }
void update(Node* n) { n->height = 1 + max(height(n->left), height(n->right)); }

void printTree(Node* n, int depth = 0) {          // sideways: right on top
    if (!n) return;
    printTree(n->right, depth + 1);
    cout << string(depth * 6, ' ') << n->key << " (BF=" << bf(n) << ")\n";
    printTree(n->left, depth + 1);
}

Node* rotateRight(Node* y) {
    Node* x = y->left;
    y->left = x->right;
    x->right = y;
    update(y); update(x);
    return x;
}

Node* rotateLeft(Node* x) {
    Node* y = x->right;
    x->right = y->left;
    y->left = x;
    update(x); update(y);
    return y;
}

Node* insert(Node* node, int key) {
    if (!node) return new Node(key);
    if (key < node->key)      node->left  = insert(node->left, key);
    else if (key > node->key) node->right = insert(node->right, key);
    else return node;

    update(node);
    int b = bf(node);

    if (b == -2 && bf(node->right) == 1) {         // RL case
        Node* z = node;
        Node* y = z->right;
        Node* x = y->left;
        cout << "\n*** RL imbalance at node " << z->key << " ***\n";
        cout << "Before: BF(" << z->key << ")=" << bf(z)
             << ", BF(" << y->key << ")=" << bf(y)
             << ", BF(" << x->key << ")=" << bf(x) << "\n";
        printTree(z);

        cout << "\nStep 1: Right-rotate " << y->key << "\n";
        z->right = rotateRight(y);
        printTree(z);

        cout << "\nStep 2: Left-rotate " << z->key << "\n";
        Node* root = rotateLeft(z);
        printTree(root);

        cout << "After: BF(" << x->key << ")=" << bf(x)
             << ", BF(" << z->key << ")=" << bf(z)
             << ", BF(" << y->key << ")=" << bf(y) << "\n";
        return root;
    }
    if (b == -2) {                                 // RR case
        cout << "\n[RR case at " << node->key << "]\n";
        return rotateLeft(node);
    }
    if (b == 2 && bf(node->left) >= 0) {           // LL case
        cout << "\n[LL case at " << node->key << "]\n";
        return rotateRight(node);
```

```
    }
    if (b == 2) {                                  // LR case
        cout << "\n[LR case at " << node->key << "]\n";
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }
    return node;
}

void runTest(const string& name, int keys[], int n) {
    cout << "\n==================== " << name << " ====================\nInsert: ";
    for (int i = 0; i < n; i++) cout << keys[i] << " ";
    cout << "\n";
    Node* root = nullptr;
    for (int i = 0; i < n; i++) root = insert(root, keys[i]);
    cout << "\nFinal tree:\n";
    printTree(root);
}

int main() {
    int t1[] = {10, 30, 20};
    int t2[] = {10, 5, 30, 20, 40, 15};
    runTest("Test 1", t1, 3);
    runTest("Test 2", t2, 6);
    return 0;
}
```

6. Program Output

Test 1: 10, 30, 20 | Test 2: 10, 5, 30, 20, 40, 15. Trees are printed sideways (right subtree on top, root at left).

```
=================== Test 1 ====================
Insert: 10 30 20

*** RL imbalance at node 10 ***
Before: BF(10)=-2, BF(30)=1, BF(20)=0
      30 (BF=1)
            20 (BF=0)
10 (BF=-2)

Step 1: Right-rotate 30
            30 (BF=0)
      20 (BF=-1)
10 (BF=-2)

Step 2: Left-rotate 10
      30 (BF=0)
20 (BF=0)
      10 (BF=0)
After: BF(20)=0, BF(10)=0, BF(30)=0

Final tree:
      30 (BF=0)
20 (BF=0)
      10 (BF=0)

=================== Test 2 ====================
Insert: 10 5 30 20 40 15

*** RL imbalance at node 10 ***
Before: BF(10)=-2, BF(30)=1, BF(20)=1
            40 (BF=0)
      30 (BF=1)
            20 (BF=1)
                  15 (BF=0)
10 (BF=-2)
      5 (BF=0)

Step 1: Right-rotate 30
                  40 (BF=0)
            30 (BF=-1)
      20 (BF=-1)
            15 (BF=0)
10 (BF=-2)
      5 (BF=0)

Step 2: Left-rotate 10
            40 (BF=0)
      30 (BF=-1)
20 (BF=0)
            15 (BF=0)
      10 (BF=0)
            5 (BF=0)
After: BF(20)=0, BF(10)=0, BF(30)=-1

Final tree:
            40 (BF=0)
      30 (BF=-1)
20 (BF=0)
            15 (BF=0)
      10 (BF=0)
            5 (BF=0)
```